GENERATIVE AI & AGENTIC AI · LESSON 5 OF 11

Retrieval-Augmented Generation (RAG): Build, Evaluate and Improve a Q&A Assistant

Ground an LLM in your own documents. RAG architecture, document loading and chunking, indexing, hybrid retrieval and re-ranking, prompt assembly with citations, answer generation (offline simulator plus real Claude code with the citations feature), evaluation of retrieval and answers, failure modes and advanced patterns (query rewriting, contextual retrieval, metadata filters, agentic and graph RAG), built end to end on a retailer's policy handbook.

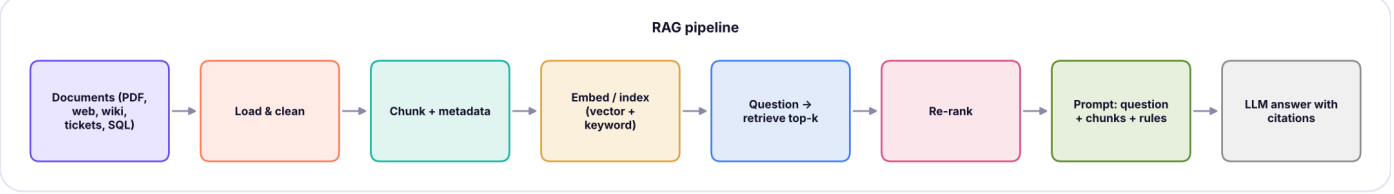
GenAI Launchpad · ✦ 2026 practice: hybrid retrieval, re-ranking, citations, contextual retrieval · Intermediate · ⏱ 140 min

By the end of this lesson you can

- Explain why and when RAG beats fine-tuning or long prompts
- Chunk documents with metadata and build a searchable index
- Retrieve with hybrid search, re-rank, and assemble a grounded prompt with citations
- Evaluate retrieval (hit rate, MRR) and answers (correctness, faithfulness, citation accuracy)
- Diagnose RAG failures and apply advanced techniques

Why RAG?

LLMs do not know your company's policies, product catalogue or yesterday's report, and they may hallucinate. **Retrieval-augmented generation** retrieves the most relevant passages for each question and gives them to the model with an instruction to answer **only from them, with citations**.



RAG has an offline indexing pipeline and an online question-answering pipeline.

Approach	Good for	Limits
Put everything in the prompt (long context)	A few documents, one-off analysis	Cost and latency grow with every call; no access control
RAG	Large, changing knowledge; citations; access control	Retrieval quality limits answer quality
Fine-tuning	Style, format, narrow skills	Poor at injecting facts; costly to update
Tools / SQL	Live numbers and transactions	Needs structured sources

The knowledge base

Our example: **NovaMart**, an online electronics retailer. The assistant answers customer-support questions from the policy handbook.

Python

```python
import re, numpy as np, pandas as pd
DOCS = {
"returns.md": """# Returns policy
## Return window
Most products can be returned within 10 days of delivery. Mobiles and laptops have a 7-day replacement-only window for manufacturing defects.
## Conditions
Products must be unused, with original box, accessories and invoice. Opened earbuds and personal-care items cannot be returned for hygiene reasons.
## Refund timeline
Refunds are issued to the original payment method within 5-7 working days after the returned product passes quality check. UPI refunds usually arrive within 48 hours.""",
"warranty.md": """# Warranty
## Standard warranty
All electronics carry the manufacturer's warranty: 1 year for mobiles, laptops and smartwatches, 6 months for chargers and cables.
## Extended warranty
NovaMart Care adds 1 or 2 extra years and covers accidental liquid damage once per year. It must be bought within 30 days of delivery.
## How to claim
Book a claim in the app under Orders → Help. A technician visit is scheduled within 72 hours in metro cities and 5 days elsewhere.""",
"delivery.md": """# Delivery
## Delivery times
Metro cities: 1-2 days. Other cities: 3-5 days. Remote pin codes: up to 8 days.
## Delivery charges
Delivery is free on orders above ₹499. Below that a ₹49 fee applies. Same-day delivery costs ₹99 in Bengaluru, Delhi NCR, Mumbai and Pune.
## Damaged parcels
If the box is damaged on arrival, refuse the delivery or report it within 48 hours with photos for a free replacement.""",
"payments.md": """# Payments
## Methods
We accept UPI, credit and debit cards, net banking, NovaMart wallet and cash on delivery up to ₹10,000.
## No-cost EMI
No-cost EMI is available on orders above ₹3,000 with selected bank cards for 3, 6 or 9 months. The interest is given back as an instant discount.
## Failed payments
If money is debited but the order fails, it is automatically refunded within 3 working days.""",
"exchange.md": """# Exchange offers
## Old device exchange
Customers can exchange an old phone or laptop for an instant discount. The final value is confirmed by the delivery agent after a physical check.
## Eligibility
The device must switch on, have no cracked screen and match the declared model. Exchange is not available with cash on delivery.""",
"loyalty.md": """# NovaMart Plus
## Benefits
Plus members get free same-day delivery, early access to sales and 5% back as NovaCoins on
```

```
every order.
## Cost
Membership costs ₹399 per year or ₹49 per month and can be cancelled any time; unused
months are not refunded.""",
}
print(len(DOCS), "documents,", sum(len(t.split()) for t in DOCS.values()), "words")
```

OUTPUT

```
6 documents, 404 words
```

Step 1: chunking

Split documents into passages small enough to be specific but large enough to carry context. Splitting on **structure** (headings, paragraphs) beats fixed character windows. Keep metadata for filtering and citations.

Python

```
def chunk_markdown(name, text, max_words=80):
    title = re.search(r"^# (.+)$", text, re.M).group(1); chunks = []
    for sec in re.split(r"^## ", text, flags=re.M)[1:]:
        heading, body = sec.split("\n", 1); words = body.split()
        for i in range(0, len(words), max_words - 15):                # 15-word overlap if
a section is long
            chunks.append({"id": f"{name}#{heading.strip().lower().replace(' ', '-')}" +
(f"-{i}" if i else ""), "doc": name, "title": title, "section": heading.strip(),
                           "text": f"{title} > {heading.strip()}: " + " ".join(words[i:i +
max_words])})
    return chunks
chunks = pd.DataFrame([c for n, t in DOCS.items() for c in chunk_markdown(n, t)])
print(len(chunks), "chunks"); print(chunks[["id", "text"]].head(4).to_string(index=False,
max_colwidth=90))
```

OUTPUT

```
16 chunks
                            id
text
     returns.md#return-window Returns policy > Return window: Most products can be
returned within 10 days of deliver...
        returns.md#conditions Returns policy > Conditions: Products must be unused,
with original box, accessories an...
   returns.md#refund-timeline Returns policy > Refund timeline: Refunds are issued to
the original payment method wit...
warranty.md#standard-warranty Warranty > Standard warranty: All electronics carry the
manufacturer's warranty: 1 year...
```

Prefixing each chunk with its document title and section ("Returns policy > Refund timeline: ...") is a cheap version of **contextual retrieval**: the chunk still makes sense on its own.

Chunking choice	Typical value	Note
Size	200–800 tokens	Smaller = precise, larger = more context
Overlap	10–20%	Avoid cutting answers in half
Boundaries	Headings, paragraphs, sentences	Use document structure
Metadata	Source, section, date, product, access group	Filters and citations

Step 2: index and retrieve (hybrid)

Python

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
word_vec = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True,
stop_words="english").fit(chunks.text)
char_vec = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5),
sublinear_tf=True).fit(chunks.text)     # robust to typos and word forms
Wd, Cd = word_vec.transform(chunks.text), char_vec.transform(chunks.text)
def retrieve(question, k=3, doc_filter=None):
    s = 0.6 * cosine_similarity(word_vec.transform([question]), Wd)[0] + 0.4 *
cosine_similarity(char_vec.transform([question]), Cd)[0]
    if doc_filter: s = np.where(chunks.doc.isin(doc_filter), s, -1)
    top = np.argsort(-s)[:k]
    return chunks.iloc[top].assign(score=s[top].round(3))
print(retrieve("How long does a refund take if I paid by UPI?")[["id",
"score"]].to_string(index=False))
print(retrieve("is there emi without interest")[["id", "score"]].to_string(index=False))
```

OUTPUT

```
                        id  score
 returns.md#refund-timeline  0.221
        payments.md#methods  0.088
payments.md#failed-payments  0.061
                        id  score
    payments.md#no-cost-emi  0.328
   warranty.md#how-to-claim  0.033
delivery.md#damaged-parcels  0.025
```

Here keyword (word) and character n-gram similarity stand in for the keyword + embedding hybrid of lesson 4; in production use BM25 plus an embedding model, and a **re-ranker** (a cross-encoder or a small LLM call) to reorder the top 20–50 candidates.

Step 3: build the grounded prompt

Python

```python
SYSTEM = """You are NovaMart's customer-support assistant.
Answer ONLY from the documents provided. If they do not contain the answer, say "I don't have that information" and suggest contacting support.
Cite the document id in square brackets after each fact, like [returns.md#refund-timeline].
Keep answers under 80 words."""
def build_prompt(question, k=3):
    ctx = retrieve(question, k)
    docs = "\n".join(f'<document id="{r.id}">\n{r.text}\n</document>' for r in ctx.itertuples())
    return f"<documents>\n{docs}\n</documents>\n\n<question>{question}</question>", ctx
prompt, ctx = build_prompt("My earbuds box was damaged when it arrived. What should I do?")
print(prompt)
```

OUTPUT

```
<documents>
<document id="delivery.md#damaged-parcels">
Delivery > Damaged parcels: If the box is damaged on arrival, refuse the delivery or report it within 48 hours with photos for a free replacement.
</document>
<document id="returns.md#conditions">
Returns policy > Conditions: Products must be unused, with original box, accessories and invoice. Opened earbuds and personal-care items cannot be returned for hygiene reasons.
</document>
<document id="warranty.md#extended-warranty">
Warranty > Extended warranty: NovaMart Care adds 1 or 2 extra years and covers accidental liquid damage once per year. It must be bought within 30 days of delivery.
</document>
</documents>

<question>My earbuds box was damaged when it arrived. What should I do?</question>
```

Step 4: generate the answer

With a real model:

Python

```
import anthropic
client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-opus-5-5", max_tokens=2000, system=SYSTEM,
    output_config={"effort": "low"},                    # a short grounded answer does not
need deep reasoning
    messages=[{"role": "user", "content": prompt}],
)
answer = next(b.text for b in response.content if b.type == "text")

# Or let the API produce verifiable citations: pass chunks as document blocks with
citations enabled
content = [{"type": "document", "source": {"type": "text", "media_type": "text/plain",
"data": r.text}, "title": r.id, "citations": {"enabled": True}} for r in ctx.itertuples()]
content.append({"type": "text", "text": question})
response = client.messages.create(model="claude-opus-5-5", max_tokens=2000, messages=
[{"role": "user", "content": content}])
for block in response.content:                           # text blocks carry .citations
with the exact cited text
    print(block.text, [c.cited_text for c in (getattr(block, "citations", None) or [])])
```

To practise offline, an **extractive** stand-in picks the retrieved sentences that best match the question, with citations. It cannot reason, but it shows the grounding and citation mechanics.

Python

```
def answer_offline(question, k=3, min_score=0.12):
    ctx = retrieve(question, k)
    if ctx.score.max() < min_score: return "I don't have that information. Please contact
NovaMart support.", ctx
    sents = [(s.strip(), r.id) for r in ctx.itertuples() for s in re.split(r"(?<=
[.!?])\s+", r.text.split(": ", 1)[1]) if s.strip()]
    sv = TfidfVectorizer(ngram_range=(1, 2), stop_words="english").fit([s for s, _ in
sents] + [question])
    sc = cosine_similarity(sv.transform([question]), sv.transform([s for s, _ in sents]))
[0]
    best = [sents[i] for i in np.argsort(-sc)[:2] if sc[i] > 0]
    return " ".join(f"{s} [{cid}]" for s, cid in best) or "I don't have that information.",
ctx
for q in ["My earbuds box was damaged when it arrived. What should I do?", "How long does a
refund take if I paid by UPI?",
          "Can I use cash on delivery with an exchange offer?", "Do you deliver to Sri
Lanka?"]:
    a, _ = answer_offline(q); print(f"Q: {q}\nA: {a}\n")
```

OUTPUT

```
Q: My earbuds box was damaged when it arrived. What should I do?
A: If the box is damaged on arrival, refuse the delivery or report it within 48 hours
```

```
with photos for a free replacement. [delivery.md#damaged-parcels] Opened earbuds and
personal-care items cannot be returned for hygiene reasons. [returns.md#conditions]

Q: How long does a refund take if I paid by UPI?
A: UPI refunds usually arrive within 48 hours. [returns.md#refund-timeline] We accept
UPI, credit and debit cards, net banking, NovaMart wallet and cash on delivery up to
₹10,000. [payments.md#methods]

Q: Can I use cash on delivery with an exchange offer?
A: Exchange is not available with cash on delivery. [exchange.md#eligibility] We
accept UPI, credit and debit cards, net banking, NovaMart wallet and cash on delivery
up to ₹10,000. [payments.md#methods]

Q: Do you deliver to Sri Lanka?
A: I don't have that information. Please contact NovaMart support.
```

The last question has no answer in the documents, and the assistant says so instead of guessing: an essential behaviour to test.

Step 5: evaluate

Build a test set of questions with the document section that contains the answer and the key fact the answer must state.

Python

```
TEST = [
    ("How many days do I have to return a speaker?", "returns.md#return-window", "10
days"),
    ("Can I return opened earbuds?", "returns.md#conditions", "cannot be returned"),
    ("When will my UPI refund arrive?", "returns.md#refund-timeline", "48 hours"),
    ("How long is the warranty on a charger?", "warranty.md#standard-warranty", "6
months"),
    ("Does NovaMart Care cover water damage?", "warranty.md#extended-warranty", "liquid
damage"),
    ("What is the delivery fee for a ₹300 order?", "delivery.md#delivery-charges", "₹49"),
    ("Is same-day delivery available in Pune?", "delivery.md#delivery-charges", "Pune"),
    ("Is no cost emi available on a 2500 rupee order?", "payments.md#no-cost-emi", "above
₹3,000"),
    ("Money was debited but my order failed", "payments.md#failed-payments", "3 working
days"),
    ("Can I exchange a phone with a cracked screen?", "exchange.md#eligibility", "no
cracked screen"),
    ("How much does Plus membership cost?", "loyalty.md#cost", "₹399"),
    ("What are the benefits of NovaMart Plus?", "loyalty.md#benefits", "same-day"),
]
rows = []
for q, gold, fact in TEST:
    ret = retrieve(q, k=3).id.tolist(); ans, _ = answer_offline(q)
    rows.append({"question": q[:42], "hit@1": gold == ret[0], "hit@3": gold in ret, "rank":
(ret.index(gold) + 1) if gold in ret else None,
                 "fact_in_answer": fact.lower() in ans.lower(), "cites_gold": gold in ans})
ev = pd.DataFrame(rows); print(ev.to_string(index=False))
mrr = np.mean([1 / r if r else 0 for r in ev["rank"]])
print(f"\nretrieval: hit@1 {ev['hit@1'].mean():.0%} | hit@3 {ev['hit@3'].mean():.0%} | MRR
{mrr:.2f} || answers: key fact present {ev.fact_in_answer.mean():.0%} | cites the right
section {ev.cites_gold.mean():.0%}")
```

OUTPUT

```
                                  question  hit@1  hit@3  rank  fact_in_answer
cites_gold
How many days do I have to return a speake   True   True     1           False
False
              Can I return opened earbuds?   True   True     1            True
True
           When will my UPI refund arrive?   True   True     1            True
True
    How long is the warranty on a charger?   True   True     1            True
True
    Does NovaMart Care cover water damage?   True   True     1            True
True
What is the delivery fee for a ₹300 order?   True   True     1            True
True
   Is same-day delivery available in Pune?   True   True     1            True
```

```
True
Is no cost emi available on a 2500 rupee o   True   True     1            True
True
     Money was debited but my order failed   True   True     1            True
True
Can I exchange a phone with a cracked scre   True   True     1            True
True
       How much does Plus membership cost?   True   True     1           False
False
   What are the benefits of NovaMart Plus?   True   True     1            True
True

retrieval: hit@1 100% | hit@3 100% | MRR 1.00 || answers: key fact present 83% |
cites the right section 83%
```

Retrieval is perfect on this small handbook, yet the crude extractive answerer misses the key fact for two questions: it picks a less relevant sentence from the right section. That is why you **measure the layers separately**: here the fix belongs in generation (a real LLM, or a better sentence scorer), not in retrieval.

Layer	Metric	How
Retrieval	Hit rate@k, MRR, recall	Labelled question → relevant chunk pairs
Answer correctness	Key facts present, exact match, human or LLM-judge score	Gold answers or checklists
Faithfulness	Every claim supported by retrieved text	LLM judge or NLI model; citation checks
Abstention	Says "don't know" for unanswerable questions	Include such questions in the test set
Operations	Latency, cost per answer, user feedback	Logs and dashboards

Frameworks such as **Ragas**, **TruLens** and **DeepEval** automate these checks; lesson 9 covers LLM-as-judge evaluation.

Failure modes and fixes

Symptom	Likely cause	Fix
Right document exists but is not retrieved	Vocabulary mismatch, bad chunking	Hybrid search, better embeddings, contextual chunk headers, query rewriting
Retrieved but answer still wrong	Too many/irrelevant chunks, weak prompt	Re-rank, fewer and better chunks, clearer instructions
Answer adds facts not in documents	Model falls back on prior knowledge	"Only from documents" + citations + faithfulness checks
Stale or conflicting answers	Old documents indexed	Metadata dates, re-index pipeline, prefer latest version
Leaks confidential content	No access control	Filter chunks by user permissions **before** prompting
Multi-part questions half-answered	Single retrieval	Decompose the question; agentic RAG (retrieve per sub-question)

Advanced patterns

- **Query rewriting / HyDE:** ask the LLM to rewrite the question (or draft a hypothetical answer) before retrieval.
- **Contextual retrieval:** prepend an LLM-written one-line context to each chunk before indexing (Anthropic reported large drops in retrieval failures, especially combined with BM25 and re-ranking).
- **Metadata filtering:** product, region, date, access group.
- **Parent-document retrieval:** search small chunks, return their larger parent section.
- **Agentic RAG:** the model decides when and what to search (tool use, lessons 6–7).
- **GraphRAG:** build a knowledge graph of entities and relations for questions that span many documents.
- **Structured + unstructured:** route numeric questions to SQL (text-to-SQL, lesson 11) and policy questions to documents.

Frameworks: LlamaIndex and LangChain (loaders, splitters, retrievers), Haystack; managed options include cloud "knowledge base" services and vector databases with built-in hybrid search.

Summary

KEY TAKEAWAYS

- RAG retrieves relevant passages and asks the model to answer only from them, with citations: up-to-date, grounded and auditable answers.
- Chunk along document structure, keep metadata, and add context headers to chunks.
- Use hybrid retrieval and re-ranking; filter by access rights before prompting.
- Instruct the model to cite sources and to say "I don't know" when the documents lack the answer; use the API's citation features where available.
- Evaluate retrieval (hit rate, MRR) and answers (correctness, faithfulness, abstention) on a labelled test set, then improve the weakest layer.

Quick quiz

6 questions · tap an answer to check it

1. What problem does RAG mainly solve?

A Giving the model up-to-date, private knowledge and grounding answers in sources

B Making models faster

C Training a new model

D Generating images

Answer: A. Retrieval supplies the facts.

2. Why add the document title and section to each chunk?

- **A** So each chunk is understandable on its own, which improves retrieval and citations
- **B** To make chunks longer
- **C** To confuse the retriever
- **D** Because embeddings require it

Answer: A. A cheap form of contextual retrieval.

3. A user asks something the documents do not cover. Ideal behaviour?

- **A** Say the information is not available and point to support
- **B** Guess an answer
- **C** Answer from general knowledge without saying so
- **D** Return an error page

Answer: A. Test abstention explicitly.

4. Which metric checks whether the answer is supported by the retrieved text?

- **A** Hit rate
- **B** Latency
- **C** Faithfulness (groundedness)
- **D** Token count

Answer: C. Every claim should trace to a source.

5. The right chunk is retrieved but ranked 9th of 10. What helps most?

A A re-ranker, or fewer, better-ranked chunks

B A higher temperature

C Larger chunks only

D Removing citations

Answer: A. Re-rankers reorder candidates precisely.

6. Where should access control be enforced?

A In retrieval: filter chunks by the user's permissions before they reach the prompt

B In the prompt text only

C After the answer is shown

D Not needed

Answer: A. The model cannot leak what it never sees.

EXERCISE

Practice

(1) Add a `shipping-abroad.md` document and check that the "Sri Lanka" question is now answered. (2) Change `max_words` to 30 and 200: how do hit@1 and the fact score change? (3) Add three unanswerable questions to the test set and measure abstention. (4) With a real API key, replace `answer_offline` with Claude and compare the fact and citation scores.

Show solution (worked answer)

Python

```
DOCS["shipping-abroad.md"] = "# International shipping\n## Countries\nWe currently
ship only within India. International delivery is not available."
chunks = pd.DataFrame([c for n, t in DOCS.items() for c in chunk_markdown(n, t)])
word_vec = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True,
stop_words="english").fit(chunks.text); char_vec = TfidfVectorizer(analyzer="char_wb",
ngram_range=(3, 5), sublinear_tf=True).fit(chunks.text)
Wd, Cd = word_vec.transform(chunks.text), char_vec.transform(chunks.text)
print(answer_offline("Do you deliver to Sri Lanka?")[0])
unanswerable = ["Do you sell cars?", "What is the CEO's phone number?", "Can I pay in
bitcoin?"]
print([answer_offline(q)[0].startswith("I don't") for q in unanswerable])
```

INTERVIEW CORNER

Questions you may be asked

Q: Explain RAG and when you would use it instead of fine-tuning. RAG retrieves relevant documents at question time and has the model answer from them; use it when knowledge is large, private or changing and you need citations. Fine-tuning suits behaviour, style or format, not frequently updated facts.

Q: How do you evaluate a RAG system? Separately: retrieval (hit rate, MRR, recall on labelled questions) and generation (correctness, faithfulness to sources, citation accuracy, abstention), plus latency and cost; use human review and LLM judges calibrated against humans.

Q: Your RAG bot gives wrong answers. How do you debug it? Inspect retrieved chunks for failing questions: if the right chunk is missing, fix chunking, retrieval or query rewriting; if present but the answer is wrong, re-rank, reduce noise, and tighten the prompt; add the cases to the test set.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.